Fall 2026 CSC 582, Assignment #1

Object-Oriented Programming of a System: The Four-Bit Adder and Extensions

Read first. Individual work, in **any object-oriented language you like**, culminating in a **monitored in-class session** one week from today — the *extensions* of the title — worth 25% of the grade. AI coding agents are permitted during the week, with disclosure (§8); not during the session (§9).

1 Deliverables and grading

What you are graded on, first, so you can plan the week around it. Your source and header files must carry JavaDoc-style comments for each class and member function, and you must run your style tool in default mode on all of them before committing any update.

Component	Weight
1. Syntactically correct, runnable code	8%
2. Incremental commit history — $\geq$15 commits over $\geq$4 distinct days, style tool run before each, every commit carrying a real message	8%
3. `AI_USAGE.md` disclosure log, kept current (§8)	5%
4. Documentation — doc comments on every class and member; generated docs output committed	8%
5. Design documentation — UML class diagram and $\geq$2 sequence diagrams (§7)	10%
6. Class and object structure — correct is-a/part-of hierarchy, `AbstractDevice` contract honored, primitives-only rule respected (§5)	14%
7. Complete truth-table unit tests for every class, gates through adder (adder level looped and exhaustive, not hand-enumerated)	15%
8. Program correctness — all tests pass, correct output	7%
9. In-class Part A — questions about your submitted work (§9)	7%
10. In-class Part B — architectural description and updated diagram (12%), plus working implementation as far as the session allowed (6%)	18%

A quarter of the grade is the part nobody else, and no tool, can do for you. Build something you understand, not just something that passes.

2 Introduction

Assignment #1 introduces you to: (1) Git,[1] used as it is on a real project; (2) an automated style tool, such as astyle;[2] (3) a documentation generator, such as Doxygen;[3] (4) a unit-testing framework, such as Google Test;[4] (5) UML modeling of structure and behavior; (6) disclosed use of AI coding agents; and (7) the use of primitives in a system implementation.

You will write a **virtual four-bit adder circuit** using only the logic gates specified below. As computer hardware is inherently complex (as defined by Booch), it possesses a natural hierarchy of components. It will be up to you to translate both its "is a" and "part of" hierarchies (known as the *canonical view*) into code.

[1] `https://git-scm.com/`

[2] `https://astyle.sourceforge.net/astyle.html`

[3] `https://www.doxygen.nl/index.html`

[4] `https://github.com/google/googletest/blob/main/docs/primer.md`

There are no teams this term, because everyone's implementation is defended live in class (§9): a finished repository is by itself weak evidence that *you* understand object-oriented design.

3 Language, and the pseudocode starter

You are **not required to use C++**. Use any language supporting classical object-oriented programming: abstract classes or interfaces with polymorphic dispatch, inheritance, and object composition. C++, Java, Python, JavaScript, C#, Kotlin, TypeScript, and Swift all qualify; plain C, purely functional languages, and Rust's struct+trait model do not. Ask before spending a week on anything you are unsure about.

The starter package contains a **pseudocode outline**: the class hierarchy, file layout, method contracts, and one worked example of each kind of unit test. It is deliberately not written in any real language, so that none is treated as the default. **What you submit must be written in a real programming language, must actually build and run on a computer, and must have a real test suite that actually executes.** Pseudocode is not a deliverable, and neither is code that was translated but never run. Setting up your own build, test framework, documentation generator, and style tool is part of the assignment.

4 Background

4.1 Primitives

Our system primitives are the logic gates AND, OR, and NAND (Fig. 1).[5] You will make a NOT gate from a NAND primitive, simply by using the same value for both of the NAND gate's inputs.

Everything else in the system — *everything*, up to and including the four-bit adder — must be built by composing instances of these three classes. You may not write raw boolean expressions anywhere outside the `update()` implementations of AND, OR, and NAND themselves.

AND – If (A = 1 AND B = 1) THEN OUT = 1

OR - If (A = 1 OR B = 1) THEN OUT = 1

NOT – if (A = 1) THEN OUT = 0

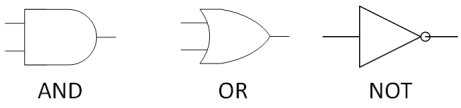

Figure 1: Basic Boolean operations.

4.2 Half-adder and full-adder

Using the basic gates we can build a half-adder (Fig. 2), whose Sum output is an XOR built from two NOTs, two ANDs, and an OR, and whose Carry output is a single AND. Build that XOR pattern as its own reusable class rather than wiring it inline — it is a distinct level of organization, and it deserves its own tests.

Figure 3 gives the complete truth table of the full-adder, and Figure 4 shows its circuit: nothing more than two cascaded half-adders, with an OR gate combining the two carries.

4.3 The four-bit adder

Connecting four full-adder circuits gives a four-bit adder (Fig. 5): the carry-out of bit i feeds the carry-in of bit $i+1$. This extends naturally to any n-bit adder, but this assignment specifies four bits only — you are graded on the *shape* of the object hierarchy, not volume of code. A composition that generalizes cleanly is worth far more here than four stanzas of copy-paste, and §9 is where that shows up.

[5] Figures 1, 2, 4, and 5 are from `https://www.waitingforfriday.com/?p=529`, unless otherwise stated.

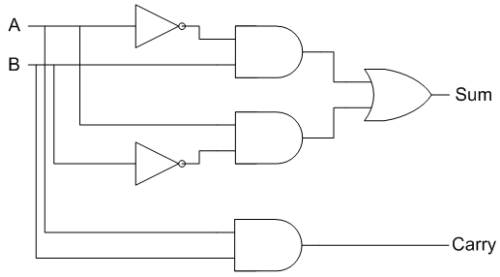

Figure 2: A half-adder circuit.

Inputs			Outputs	
A	B	C – IN	Sum	C - Out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Figure 3: The full-adder truth table (from `https://www.geeksforgeeks.org/full-adder-in-digital-logic/`).

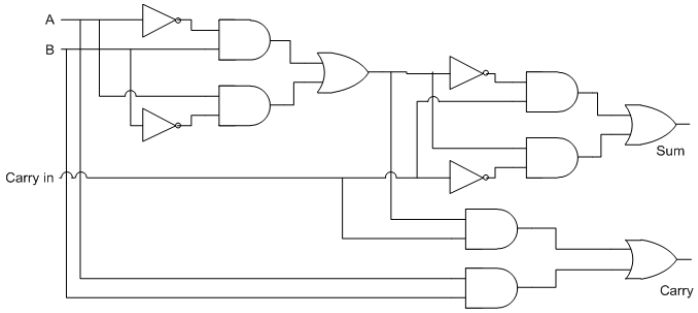

Figure 4: A full-adder circuit.

5 Required class hierarchy

`AbstractDevice` (abstract) — `update()` recomputes outputs; every setter on every device below must trigger it.
`AbstractGate` (abstract, extends `AbstractDevice`) — the 2-in/1-out gate contract.
`AND`, `OR`, `NAND` (extend `AbstractGate`) — the only primitives; the only place real boolean logic appears.

Composition only below this line:
`Xor2` — NOT + NOT + AND + AND + OR
`HalfAdder` — 1×`Xor2` + 1×`AND` → Sum, Carry
`FullAdder` — 2×`HalfAdder` + 1×`OR` → Sum, Cout
`FourBitAdder` — 4×`FullAdder`, ripple carry

You must create the `AbstractDevice` class yourself; there is no placeholder for it in the starter. It exists to enforce the updating of the output pins whenever the input pins are set on any higher-order circuit in this system. The provided `AbstractGate` does **not** yet derive from anything — retrofitting it is one of your first tasks. And note that `FullAdder` and `FourBitAdder` have a different pin shape than a 2-in/1-out gate, which is exactly why `AbstractDevice` sits above `AbstractGate` rather than beside it.

6 Project structure

Keep the starter's layout when you translate it into your language, whichever language that is:

`source/FourBitAdder/FourBitAdder.<ext>` — the public entry point
`source/FourBitAdder/intern/` — `AbstractGate`, `LogicGates`, everything you build
`tests/<toolname>/` — named for your framework (`gtests/`, `junit/`, `pytest/`, `jest/`)
`design/` — your UML and sequence diagrams (§7)
`doc/` — generated documentation output, committed in your final submission
`AI_USAGE.md` — your disclosure log (§8)

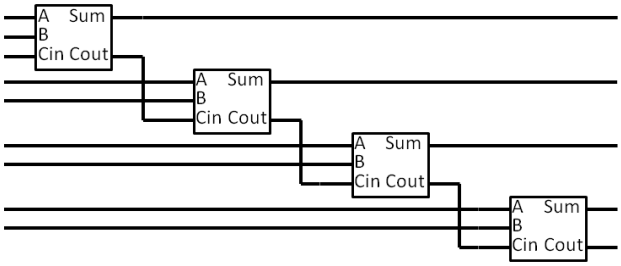

Figure 5: The four-bit adder circuit.

This organization encapsulates the implementation of `FourBitAdder` by separating it from its interface. How strictly each language enforces that boundary differs; where yours cannot, the directory split still documents the intent and you are expected to respect it. Where your language's conventions collide with this layout, follow the layout and note the deviation in your `README`. A **local** git repository is required, but no GitHub account and no hosted remote — see §10.

7 Design documentation: UML

In `design/`, include **one UML class diagram** covering every class from `AbstractDevice` through `FourBitAdder`, showing inheritance and composition relationships both. Your documentation generator may produce this for you (Doxygen with Graphviz enabled does); check that what it emits is complete rather than assuming.

Also include **at least two sequence diagrams**, hand-authored, showing behavior rather than structure: one tracing the cascade of `update()` calls when a single input pin changes on a composite device, and one tracing an addition end-to-end, including how a carry propagates from the low bit to the high bit.

Diagrams must be committed in a plain-text format — Mermaid[6] (recommended), PlantUML, or Graphviz. This is not a style preference. Your submission is a text bundle (§10), so a diagram existing only as a PNG, SVG, or drawing-tool export cannot be read by anyone grading it and scores nothing here. Drawing in a GUI tool is fine *provided* you also commit a text version. The bundler lists every file it could not include, so check that list before you submit. A class diagram can be generated; a sequence diagram of your own implementation mostly cannot, which is precisely why it is required.

8 AI coding agent policy

AI coding agents — Claude Code, GitHub Copilot, ChatGPT, Cursor, and the rest — are **permitted during the take-home week, with disclosure**. This is how software is written now, and this course is not going to pretend otherwise.

Keep `AI_USAGE.md` current as you work: which tool, roughly when, what you asked for, what it produced, and what you changed, rejected, or had to fix. A template is included in the starter package. You are responsible for every line in your repository regardless of who or what typed it; "the agent wrote it" is not a defense for code you cannot explain or extend. Using AI is not itself an integrity violation — failing to disclose it, or turning out not to understand code you submitted as your own, is. Sharing your repository or solution with another student is not permitted.

9 The in-class session

The session has two graded parts. **AI coding assistants and generative chat tools are off** for its entire duration; official language and library documentation, plus your own code, notes, and diagrams, are permitted. Bring a laptop with your submitted repository and diagrams on it.

[6] `https://mermaid.js.org/`

Part A — questions about what you submitted (7%). You will be asked, individually, about the code you submitted before class that day: why a class exists, where a behavior is implemented, what happens in what order when a method is called, what you would change to do X. These are not trick questions; a student who built the system answers them in a sentence.

Part B — a live extension task (18%). You will be handed one additional task, sized for two hours, that builds directly on the code you submitted. You will not know the specific task beforehand, but you know its shape and should design with it in mind: it will push on how well your system *generalizes* — how it would accommodate a wider datapath, retained state, or sequencing and timing, rather than one fixed combinational circuit computed in a single shot.

Part B has two deliverables, and the first matters more: (a) **a written description of the architectural changes** — which classes appear, disappear, or change responsibility, where they sit in the hierarchy, and what the change costs you in the existing design — plus an updated sequence diagram; and (b) **as much working implementation as the two hours allow**. You are not expected to finish: a coherent architecture with two classes implemented and the rest clearly specified scores better than half-wired code with no discernible design. Two hours is plenty of time to re-architect a system you built cleanly, and nowhere near enough to first understand one you did not.

10 Submission

Submission is a single text file, produced by the `munger.py` script in the starter package. *Munge* (v.): to transform data mechanically from one form into another; a *munger* is a tool that does so. This one folds your entire multi-file project — commit history, directory tree, and the contents of every source, test, config, and design file — into one `.txt` file, so that a project in any language can be read and graded uniformly. Run `python3 munger.py` from the root of your repository. It asks for your student ID, your name, and **which of the two submissions this is**, then writes either `<ID>_<Name>_takehome_bundle.txt` or `<ID>_<Name>_inclass_bundle.txt`. The starter package includes a walk-through for trying this out before submission day.

The two submissions use two different Google Forms. The take-home bundle goes here, by the start of class:

`https://forms.gle/8BxR4VmePRjMWs5s9`

The in-class bundle goes to a separate form, handed out during the session itself. Check the filename against the form before you upload.

Do not modify `munger.py`. Every bundle records the SHA-256 of the script that produced it, and that digest is checked when your submission is graded. The published digest is `5aaae171f2ee60b3e65243858cfddf72 7db996bb21077604eddefd9dda93a3e6`. Verify your copy with `sha256sum munger.py` (or `shasum -a 256 munger.py` on macOS). If the script misses something your project needs, email me — do not edit it. It remains your responsibility to open the bundle and confirm your work is in there before uploading. The bundle ends with a `FILES NOT BUNDLED` list naming everything that could not be included as text — if a diagram appears there, it will not be graded (§7).

After Part B of the in-class session, commit your work, re-run `munger.py` choosing the in-class option, and upload that bundle before time is called. It is what is graded for rubric item 10; your take-home bundle still stands for items 1–8.